

Systèmes d'exploitation I

Chapitre 2 : Processus

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

Sommaire

- 1 Introduction
- 2 Contexte, environnement
- 3 Création des processus dans Unix
- 4 Quelques interactions entre processus dans Unix

Qu'est ce qu'un processus?

Un Processus est l'entité dynamique qui représente l'exécution un programme sur un processeur

- **Programme:**

Code + data (**passif**)

```
int i;  
int main() {  
    printf("Salut\n");  
}
```

- **Processus:**

Programme **en cours d'exécution**

Stack	
Heap	
Data	int i;
Code	main()

- 2 personnes qui exécutent le même programme sur la même machine :
 - ▶ 2 processus différents.

→ Compétition

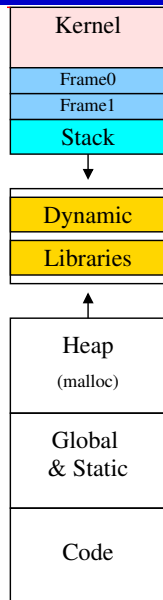
- Plusieurs processus veulent accéder à une ressource exclusive (*i.e.* ne pouvant être utilisée que par un seul à la fois) :
 - ▶ Processeur (cas du pseudo-parallélisme)
 - ▶ Imprimante, carte son
- Une **solution possible** parmi d'autres:
FIFO: premier arrivé, premier servi (les suivants **attendent** leur tour)

→ Coopération

- Plusieurs processus collaborent à une tâche commune
- Souvent, ils doivent se **synchroniser**:
 - ▶ p1 produit un fichier, p2 imprime le fichier
 - ▶ p1 met à jour un fichier, p2 consulte le fichier
- La synchronisation se ramène à :
p2 doit **attendre** que p1 ait franchi un certain point de son exécution

Processus UNIX

- Processus = exécution d'un programme
 - ▶ Commande (shell)
 - ▶ Application
- **Le contexte d'un processus** est l'ensemble des données qui le caractérise permettant de reprendre son exécution si interrompu.
 - ▶ **Contexte utilisateur** : code, data & stack (user)
 - ▶ **Contexte matériel** : l'ensemble des registres du processeur
 - ▶ **Contexte système** : les structures de données du système pour la réalisation du processus.
- Les processus sont **identifiés** par leur **pid**
 - ▶ Commande ps : liste des processus
 - ▶ Commande top : montre l'activité du processeur
 - ▶ Primitive getpid() : renvoie le pid du processus courant



Environnement d'un processus

- Ensemble de variables accessibles par le processus (sorte de configuration)
- Principaux avantages:
 - ▶ L'utilisateur n'a pas à redéfinir son contexte pour chaque programme
Nom de l'utilisateur, de la machine, terminal par défaut, ...
 - ▶ Permet de configurer certains éléments
Chemin de recherche des programmes (PATH), shell utilisé, ...
- Certaines sont prédéfinies par le système (et modifiables par l'utilisateur)
- L'utilisateur peut créer ses propres variables d'environnement
- Interface (dépend du shell):

Commande tcsh	Commande bash	Action
setenv	printenv	affiche toutes les variables définies
setenv VAR <valeur>	export VAR=<valeur>	attribue la valeur à la variable
echo \$VAR	echo \$VAR	affiche le contenu de la variable

- Exemple : `export DISPLAY=blaise.loria.fr:0.0` définit le terminal utilisé
- L'interface de programmation sera vue en TD/TP

Vie et mort des processus

Tout processus a un début et une fin

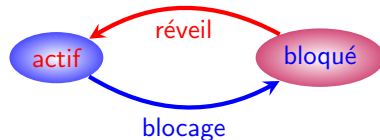
- **Début** : création par un autre processus
 - ▶ `init` est le processus originel: `pid=1` (`launchd` sous mac)
Créé par le noyau au démarrage, il lance les autres processus système
- **Fin**
 - ▶ Auto-destruction (à la fin du programme) (par `exit`)
 - ▶ Destruction par un autre processus (par `kill`)
 - ▶ Destruction par l'OS (en cas de violation de protection et autres)
 - ▶ Certains processus ne se terminent pas avant l'arrêt de la machine
 - ★ Nommés "**démons**" (disk and execution monitor → `daemon`)
 - ★ Réalisent des fonctions du système (login utilisateurs, impression, serveur web)

Création de processus dans UNIX

- Dans le langage de commande:
 - ▶ Chaque commande est exécutée dans un processus séparé
 - ▶ On peut créer des processus en (pseudo-)parallèle:
`$ prog1 & prog2 & # crée deux processus pour exécuter prog1 et prog2`
`$ prog1 & prog1 & # lance deux instances de prog1`
- Par l'API : clonage avec l'appel système `fork` (cf. transparent suivant)

Blocage de processus

→ Définition d'un nouvel état de processus: **bloqué** (exécution suspendue ; réveil explicite par un autre processus ou par le système)



Fonction `sleep(n)`

- Bloque le processus courant pour n secondes
- `unsigned int sleep(unsigned int seconds);`
- `usleep()` et `nanosleep()` offrent meilleures résolutions (micro, nanoseconde) mais interfaces plus compliquées et pas portables

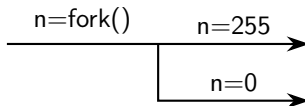
somnole: affiche à chaque seconde le temps restant à dormir

```
void somnole(unsigned int secondes) {
    int i;
    for (i=0; i<secondes; i++) {
        printf("Déjà dormi %d secondes sur %d\n", i, secondes);
        sleep(1);
    }
}
```


Création des processus dans Unix (1/3)

Appel système `pid_t fork()`

- Effet: **clone** le processus appelant
- Le processus créé (**fils**) est une copie conforme du processus créateur (**père**)
Copies conformes comme une bactérie qui se coupe en deux
- Ils se reconnaissent par la valeur de retour de `fork()`:
 - ▶ Pour le père : le `pid` du fils (ou `-1` si erreur)
 - ▶ Pour le fils : `0`



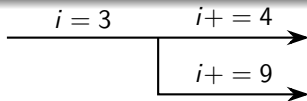
Exemple:

```
if (fork() != 0) {  
    printf("je suis le père, mon PID est %d\n", getpid());  
} else {  
    printf("je suis le fils, mon PID est %d; mon père est %d\n", getpid(), getppid());  
    /* en général exec(), (exécution d'un nouveau programme) */  
}
```

Création des processus dans Unix (2/3)

```
int i=3;
if (fork() != 0) {
    printf("je suis le père, mon PID est %d\n", getpid());
    i += 4;
} else {
    printf("je suis le fils, mon PID est %d; mon père est %d\n",
        getpid(), getppid());
    i += 9;
}
printf("pour %d, i = %d\n", getpid(), i);
```

```
je suis le fils, mon PID est 10271; mon père est 10270
pour 10271, i =
je suis le père, mon PID est 10270
pour 10270, i =
```

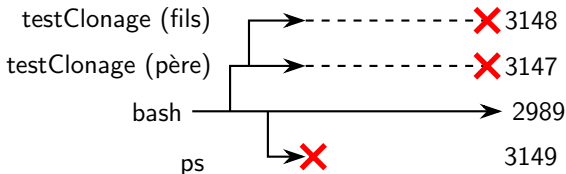


Création des processus dans Unix (3/3)

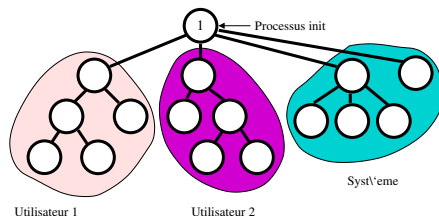
testClonage.c

```
int main() {
    if (fork() != 0) {
        printf("je suis le père, mon PID est %d\n", getpid());
        sleep(10) /* blocage pendant 10 secondes */
        exit(0);
    } else {
        printf("je suis le fils, mon PID est %d\n", getpid());
        sleep(10) /* blocage pendant 10 secondes */
        exit(0);
    }
}
```

```
$ gcc -o testClonage testClonage.c
$ ./testClonage & ps
je suis le fils, mon PID est 3148
je suis le père, mon PID est 3147
[2] 3147
  PID TTY          TIME CMD
 2989 pts/0    00:00:00 bash
 3147 pts/0    00:00:00 testClonage
 3148 pts/0    00:00:00 testClonage
 3149 pts/0    00:00:00 ps
$
```



Hiérarchie de processus Unix

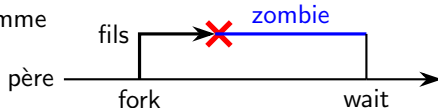
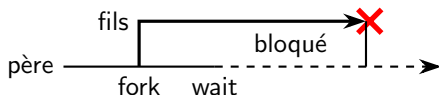


- Quelques appels systèmes utiles:

- ▶ `getpid()` : obtenir le numéro du processus
- ▶ `getppid()` : obtenir le numéro du père
- ▶ `getuid()` : obtenir le numéro d'utilisateur (auquel appartient le processus)

Synchronisation entre un processus père et ses fils

- Fin d'un processus: `exit(etat)`
etat est un code de fin (convention : 0 si ok, code d'erreur sinon – cf. `errno`)
- Le père attend la fin de l'un des fils: `pid_t wait(int *ptr_etat)`
retour : pid du fils qui a terminé; code de fin stocké dans `ptr_etat`
- Attendre la fin du fils `pid`:
`pid_t waitpid(pid_t pid, int *ptr_etat, int options)`
- Processus zombie: terminé, mais le père n'a pas appelé `wait()`.
Il ne peut plus s'exécuter, mais consomme encore des ressources. À éviter.



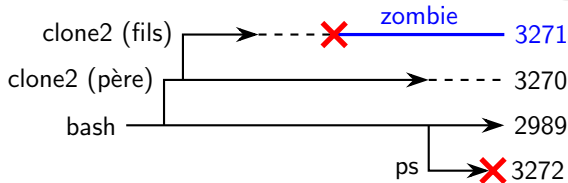
Exemple de synchronisation entre père et fils

Il y a un zombie

clone2.c

```
int main() {
  if (fork() != 0) {
    printf("je suis le père, PID : %d\n", getpid());
    while (1) ;
    /*boucle infinie sans attendre le fils*/
  } else {
    printf("je suis le fils, PID : %d\n", getpid());
    sleep(2) /* blocage pendant 2 secondes */
    printf("fin du fils\n");
    exit(0);
  }
}
```

```
$ gcc -o clone2 clone2.c
$ ./clone2
je suis le fils, PID : 3271
je suis le père, PID : 3270
fin du fils
usr : <ctrl-Z> (suspend)
Suspended
$ ps
PID TTY          TIME CMD
2989 pts/0    00:00:00 bash
3270 pts/0    00:00:03 clone2
3271 pts/0    00:00:00 clone2 <defunct>
3272 pts/0    00:00:00 ps
$
```



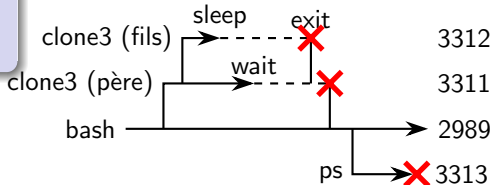
Autre exemple de synchronisation père fils

Il n'y a pas zombie

clone3.c

```
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    if (fork() != 0) {
        int statut; pid_t fils;
        printf("Le père (%d) attend.\n", getpid());
        fils = wait(&statut);
        if (WIFEXITED(statut)) {
            printf("%d : fils %d terminé (code %d)\n",
                getpid(), fils, WEXITSTATUS(statut));
        };
        exit(0);
    } else { /* correspond au if (fork() != 0) */
        printf("je suis le fils, PID=%d\n",
            getpid());
        sleep(2) /* blocage pendant 2 secondes */
        printf("fin du fils\n");
        exit(1);
    }
}
```

```
$ ./clone3
je suis le fils, PID=3312
Le père (3311) attend
fin du fils
3311: fils 3312 terminé (code 1)
$ ps
PID TTY          TIME CMD
2989 pts/0    00:00:00 bash
3313 pts/0    00:00:00 ps
$
```



Exécution d'un programme spécifié sous UNIX

Appels systèmes `exec`

- Pour faire exécuter un nouveau programme par un processus
- Souvent utilisé immédiatement après la création d'un processus:
fork+exec = lancement d'un programme dans un nouveau processus
- **Effet:** remplace la mémoire virtuelle du processus par le programme
- Plusieurs variantes existent selon le mode de passage des paramètres (tableau, liste, passage de variables d'environnement)
- C'est aussi une primitive du langage de commande (même effet)

Exemple:

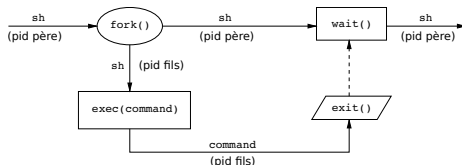
```
main() {
    if (fork() == 0) {
        code=execl("/bin/ls", "ls", "-a", 0); /* le fils exécute : /bin/ls -a .. */
        if (code != 0) { ... } /* Problème dans l'appel système; cf. valeur de errno */
    } else {
        wait(NULL); /* le père attend la fin du fils */
    }
    exit(0);
}
```


L'exemple du shell

Exécution d'une commande en premier plan

\$ commande

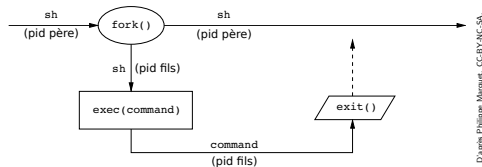
- 4 syscalls: fork, exec, exit, wait



Exécution d'une commande en tâche de fond

\$ commande &

- Le shell ne fait pas wait()
- Il n'est plus bloqué



D'après Philippe Marquet, CC-BY-NC-SA

Résumé du chapitre

- Utilité des processus
 - ▶ Simplicité (séparation entre les activités)
 - ▶ Sécurité (séparation entre les activités)
 - ▶ Efficacité (*quand l'un est bloqué, on passe à autre chose*)
- Interface UNIX
 - ▶ Création: `fork()`
 - ★ résultat=0 → je suis le fils
 - ★ résultat>0 → je suis le père (résultat=pid du fils)
 - ★ résultat<0 → erreur
 - ▶ Attendre un fils : deux manières
 - ★ `wait()`: n'importe quel fils
 - ★ `waitpid()`: un fils en particulier
 - ▶ Processus zombie : un fils terminé dont le père n'a pas fait `wait()`
 - ▶ Appel système `exec()`: remplace l'image du process actuel par le programme spécifié
- Bloquer le processus courant : deux façons
 - ▶ Jusqu'au prochain signal: `pause()`
 - ▶ Pendant 32 secondes: `sleep(32)`